

Apostila de revisão — Leilão de Cavalos

Material de revisão dos conceitos de Java usados no projeto: fundamentos, classes, objetos, encapsulamento, arrays, String, Scanner, exceções e integração entre backend e frontend.

1. Como o projeto funciona

O programa começa em `Main.main`, cria os dados iniciais, inicia a API local e exibe um menu de console. O usuário pode listar cavalos, pesquisar, iniciar leilões, cadastrar participantes, registrar lances, consultar histórico, encerrar e reabrir leilões.

O frontend consulta a API Java em `http://127.0.0.1:8080/api`. O frontend é servido separadamente em `http://127.0.0.1:8000`.

2. Classes do projeto

- **Cavalo:** guarda id, nome, raça, idade, sexo, valor inicial e situação de venda.
- **Participante:** guarda id, nome e CPF, validando o formato e os dígitos verificadores.
- **Lance:** associa um participante a um valor e registra a data.
- **Leilao:** contém um cavalo, uma lista em array de lances e as regras de lance, encerramento e reabertura.
- **LeilaoService:** administra os arrays de cavalos, participantes e leilões.

3. Encapsulamento

Os atributos são `private`. O acesso acontece pelos métodos públicos, como `getNome()`, `getValor()` e `isEncerrado()`. As regras ficam dentro das classes,

evitando estados inválidos.

4. Arrays

O serviço utiliza arrays de tamanho fixo: 50 cavalos, 100 participantes e 50 leilões. As variáveis de quantidade indicam quantas posições estão ocupadas. O primeiro índice de um array é zero.

```
for (int i = 0; i < service.getQuantidadeCavalos(); i++) {  
    Cavalo cavalo = service.getCavalos()[i];  
    System.out.println(cavalo);  
}
```

5. Scanner, condicionais e laços

`Scanner` lê as opções do menu. O `switch` escolhe a operação. O laço `do-while` mantém o menu funcionando até a opção zero. Os laços `for` percorrem os arrays.

6. Exceções

- `CavaloNaoEncontradoException` : cavalo ou leilão não encontrado.
- `LanceInvalidoException` : valor inválido, participante inexistente ou lance menor que o atual.
- `LeilaoEncerradoException` : tentativa de lance em leilão fechado.

```
try {  
    leilao.adicionarLance(lance);  
} catch (LanceInvalidoException e) {  
    System.out.println(e.getMessage());  
}
```

7. Regras de negócio

- O lance precisa ser positivo, finito e ter no máximo duas casas decimais.
- O primeiro lance precisa superar o valor inicial.
- Os próximos lances precisam superar o maior lance anterior.

- Leilões encerrados não aceitam lances.
- Reabrir o leilão mantém o histórico.
- CPF inválido ou duplicado não é aceito.

8. Métodos principais

- `LeilaoService.iniciarLeilao(id)` : busca o cavalo e cria ou recupera o leilão.
- `Leilao.adicionarLance(lance)` : valida e armazena o lance.
- `Leilao.encerrarLeilao()` : fecha o leilão e marca o cavalo como vendido quando há lance.
- `Leilao.reabrirLeilao()` : abre novamente e preserva o histórico.
- `LeilaoService.buscarCavalo(pesquisa)` : pesquisa pelo nome sem diferenciar maiúsculas e minúsculas.

9. Como executar

```
$fontes = Get-ChildItem backend/src -Recurse -Filter *.java | Select-Object -ExpandProperty FullName  
javac -encoding UTF-8 --release 17 -d backend/out $fontes  
java -cp backend/out Main
```

Em outro terminal:

```
cd frontend  
python -m http.server 8000 --bind 127.0.0.1
```

10. Perguntas para revisar

1. Qual é a diferença entre classe e objeto?
2. Por que os atributos são privados?
3. Por que o array precisa de uma variável de quantidade?
4. Qual a diferença entre `==` e `equals` ?
5. Em que situações cada exceção personalizada é lançada?
6. Como o frontend envia um lance para o backend?

Checklist: compilar com Java 17; iniciar o backend; iniciar o frontend; testar um lance válido; testar um lance menor; encerrar; reabrir; consultar o histórico.